

[유런 10주차 출석 대체과제(요약본)]

5. 토큰화

5.0. 토큰화 개요

- 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술을 자연어 처리(NLP)라고 함. 자연어 처리 모델을 만들기 위해 가공되지 않은 말뭉치(string)를 어떤 기준으로 잘라 토큰(string)으로 만들어야 함. == 토큰화
- 토큰나이저 구축 방법: 공백 분할/정규표현식/어휘 사전/머신러닝 활용 등
➔ 토큰나이저 구축 방법에 따라 시스템의 성능/처리 결과에 영향을 주기도 한다

5.1. 단어 및 글자 토큰화

- 단어 토큰화: 텍스트 데이터 -> 단어로 분리 (split)
- 글자 토큰화: 텍스트 -> 글자로 분리
 - 언어 모델링 등 시퀀스 예측 작업에서 활용함
 - jamo 라이브러리를 활용
 - 장점: 단어 사전 크기가 비교적 작음, OOV 감소
 - 단점: 개별 토큰의 의미가 없어 다의어나 동음이의어 구별이 어렵고 시퀀스의 길이가 길어지면 연산량이 증가함

5.2. 형태소 토큰화

: 텍스트 -> 형태로 단위로 분리

- 언어의 문법과 구조를 고려해 단어를 분리하기 때문에 의미 반영도 가능
- 형태소 어휘 사전: 단어가 어떤 형태소들의 조합으로 이루어져 있는지 / 각 형태소가 어떤 품사에 속하는지 / 해당 품사 뜻
- 품사 태깅: 텍스트 데이터 -> 형태소 분석 -> 해당 품사 태깅

5.2.1. KoNLPy: 한국어 자연어 처리를 위해 개발된 라이브러리

- 명사 추출, 형태소 분석, 품사 태깅 등의 기능
- 형태소 분석기 Okt, Kkma, Komoran
 - Okt -> okt.nouns / okt.phrases / okt.morphs / okt.pos
 - Kkma -> kkma.nouns / kkma.sentences / kkma.morphs / kkma.pos

5.2.2. NLTK: (다양한 언어의) 자연어 처리를 위해 개발된 라이브러리

- 토큰화, 품사 태깅 작업을 위해 패키지나 모델을 다운로드 해야됨

- Punkt 모델: 통계 기반 모델
- Averaged Perception Tagger 모델: 퍼셉트론 기반 모델 / 35개 품사 태깅

5.2.3. spaCy: 사이언 기반 오픈소스 라이브러리로, 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반 자연어 처리 라이브러리

- NLTK는 자연어 처리 알고리즘 학습 목적, spaCy는 효율적인 처리 속도와 높은 정확도를 제공하는 것을 목적으로 하기 때문에 spaCy의 모델이 더 크고 복잡하고 리소스 요구도 큼

5.3. 하위 단어 토큰화

- 기존 형태소 분석기 문제점
 - OOV(Out-of-Vocabulary): 신규어/신조어/도메인 특화 용어 처리에 취약
 - 형태소 경계 의존: 띄어쓰기/품사 태깅 오류 전파
 - 언어/도메인 편향: 특정 코퍼스, 사전 설계에 과적합
 - 유연성 부족: 다국어/도메인 확장 시 재설계/재학습 비용이 큼
- 하위 단어 토큰화: 중간 단위로 분해하자
 - 장점: OOV 감소, 어휘 축소, 형태 변형에 대응 가능
 - 원칙: 자주 함께 나타나는 문자열을 묶어 subword 단위로 모델링
- Byte Pair Encoding

: (1) 문자 단위 시작 (2) 가장 자주 출현하는 인접 쌍 병합 반복

- 산출물: 병합 규칙(merges) + 어휘(vocab)
- 특징: 빈도 기반 병합, 간단하고 효율적, 신어 일반화에 강함

** SentencePiece: 바이트 페어 인코딩과 유사한 알고리즘을 지원하는 하위 단어 토크나이저 라이브러리

- 사용자가 직접 설정 가능한 하이퍼파라미터 제공
- 언어 비의존: 공백 비의존, raw text 바로 학습 (BPE, Unigram 알고리즘)
- 장점: 데이터 정규화 및 특수 토큰 처리 포함, 재현성이 우수

** Korpora: 한국어 공개 코퍼스 수집 패키지로 실습용 말뭉치 확보 용이

★ 토크나이저 학습 공통 프로세스

1. 데이터 수집: 도메인 대표성/크기/클린
2. 정규화: lowercasing, NFD/NFKC, 특수문자 규칙 통합
3. 프리토큰화: 공백/ByteLevel/Metaspaces 등

4. 모델 학습: BPE/WordPiece/Unigram, vocab 크기 결정해야 함
 5. 포스트프로세싱: [CLS], [SEP] 삽입 등 템플릿 처리
 6. 평가: 토큰 길이, 커버리지, OOV, Perplexity 대리 지표
 7. 저장/배포: 버전 고정, 재현성/호환성 체크
- WordPiece
 - : 빈도 기반 병합 알고리즘 BPE와 달리, 언어모델 우도/점수를 사용함

$$\text{score}(x, y) = \frac{f(x, y)}{f(x)f(y)}$$

- 점수: -> 결합 강도가 큰 쌍을 우선 병합

** Hugging Face tokenizers 라이브러리

- 특징
 - 러스트 기반 고성능, 파이썬 바인딩
 - BPE/WordPiece/Unigram/ByteLevel 등 지원
 - 정규화/프리티oken화/포스트프로세싱 모듈화
 - 불필요 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수문자 처리 등 제공
 - 공백 혹은 구두점을 기준으로 입력 문장을 나눠 텍스트 데이터를 효과적으로 처리
 - 대용량 코퍼스에서도 빠른 학습 가능
 - transformers와 호환됨 (저장 포맷 및 로더가)
- 모델 학습 과정
 1. 코퍼스 수집 및 정제
 2. 정규화 규칙 확정 (NFKC, 숫자/기호 규칙)
 3. 프리토큰화(Whitespace/ByteLevel)
 4. WordPiece 학습
 5. 포스트프로세싱([CLS], [SEP] 템플릿)
 6. 품질 평가(토큰 길이/커버리지/oov율)
 7. 버전 관리 및 배포 (json 저장)

6. 임베딩

6.0. 임베딩 개요

- 텍스트 벡터화: 텍스트 -> 숫자 (컴퓨터가 이해할 수 있게)
 - 원-핫 인코딩: 문서에 등장하는 각 단어를 고유 색인 값으로 매핑한 뒤 해당 색인 위치를 1로 표시, 나머지는 모두 0으로 표시

- 빈도 벡터화(Count): 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식
- 워드 임베딩: 단어를 고정된 길이의 실수 벡터로 표현
 - 대표 기법: Word2Vec, fastText
 - 단점: 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어려움 -> 동적 임베딩 사용

6.1. 언어모델(Language Model)

: 주어진 문장으로 문맥을 이해하고 문장 구성에 대한 예측 수행

- 자기회귀 언어 모델: 입력된 문장 조건부 확률 -> 다음 단어 예측
 - 조건부 확률:

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

- 통계적 언어 모델: 시퀀스에 대한 확률 분포를 추정해 다음에 등장할 단어의 확률 예측
 - 구현 - 마르코프 체인: 빈도 기반 조건부 확률 모델

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- GPT, BERT
- 새로운 문장을 생성하거나 다양한 종류의 텍스트 데이터 학습이 가능
- 대규모 자연어 처리에 효과적

6.2. N-gram

: 텍스트에서 N개의 연속된 단어 시퀀스가 등장할 확률 추정

- 관용적 표현 분석, 작은 규모의 데이터셋에서 연속된 문자열 패턴 분석, 자연어 처리 작업 및 문자열 패턴을 분석할 때 활용

6.3. TF-IDF

: 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, '문서 빈도'와 '역문서 빈도'를 곱한 값임

- BoW: 문서나 문장을 단어의 집합으로 표현하는 방법. 빈도 기록
- 단어 빈도 (TF): 문서 내에서 특정 단어의 빈도수 $TF(t,d) = count(t,d)$
- 문서 빈도 (DF): 한 단어가 얼마나 많은 문서에 나타나는지

$$DF(t,D) = count(t \in d: d \in D)$$

- 역문서 빈도 (IDF): 문서 내에서 특정 단어가 얼마나 중요한지

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

** TF-IDF 클래스 매개변수: input, encoding, lowercase, stop_words, ngram_range, max_df, min_df, vocabulary, smooth_idf

1. TfidfVectorizer 클래스의 객체 생성
 2. fit() 메소드 -> 말뭉치 학습 = 모델이 단어 사전 구축
 3. transform() 메소드 -> 학습된 객체로 데이터를 TF-IDF 벡터로 변환
- TF-IDF 한계점
 - 문맥, 단어 순서 고려 x
 - 단어 의미 반영 x (중요도만 나타냄)